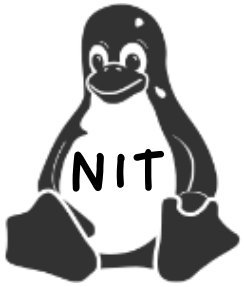
Welcome



NIT Pros!

Day 09

Let Us Learn Linux

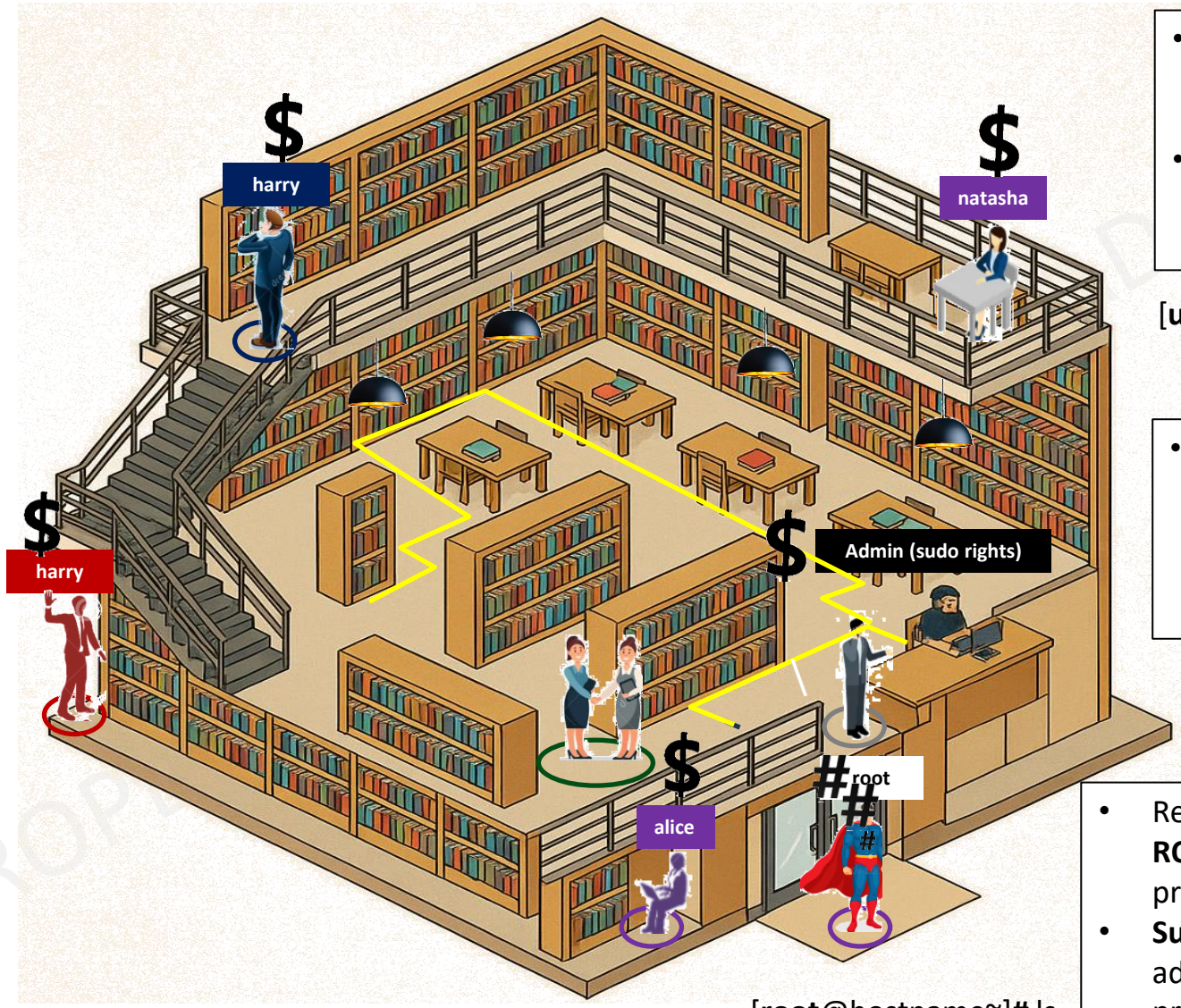
1

CHAPTER 1
Linux Filesystem

Using the rules and strategies in this section you will gain confidence and a deep insight in setting up yourself for success

- ❑ Directories
- ❑ Files
- ❑ Filesystem
- ❑ Inodes
- ❑ Important Linux Commands

LINUX FILE SYSTEM



$

- Represents a **regular USER** shell prompt
- Non-root user / **normal user** account

[**user**@hostname~]**$** ls

- SUDO is a regular user with administrative privileges

#

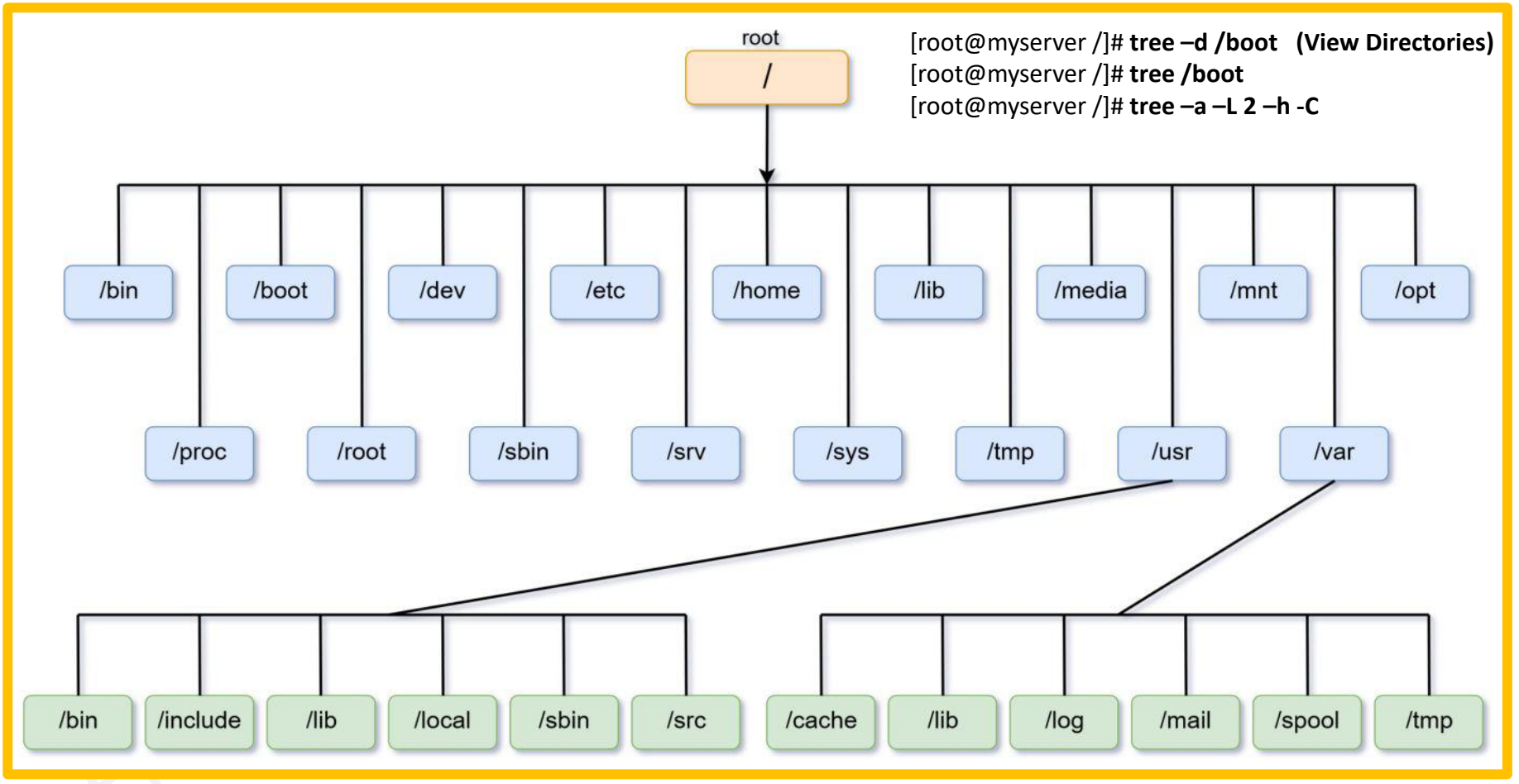
- Represents a **ROOT-USER** shell prompt
- **Superuser** with full administrative privileges

[**root**@hostname~]**#** ls

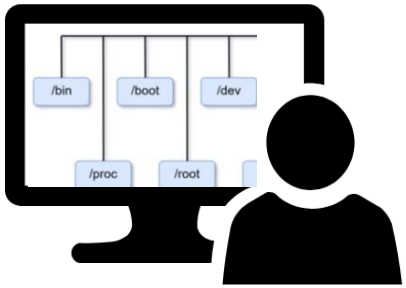
Manage Files

Date: Aug-2nd-2025

[root@myserver /]# **dnf install tree –y (let us install “tree” command)**



Linux File System Function...



(Logical File System)

[root@hostname~]# cat /proc/cpuinfo (Virtual File System)

Virtual Machine (VM)

Bash Shell

System Call

Kernel - OS

Logical File System

Virtual File System

Physical System

Linux File System

Partition 1

Partition 2

Partition 3

Hard Disk

Linux Filesystem Architecture

From configuration to storage: how Linux organizes and manages your data

1 SYSTEM CONFIGURATION

The system reads configuration from key files such as:

- /etc/resolv.conf (DNS)
- /etc/fstab (filesystems)
- /etc/hosts (host mappings)
- ...and more

/etc/resolv.conf

Configuration file

Example: DNS Settings

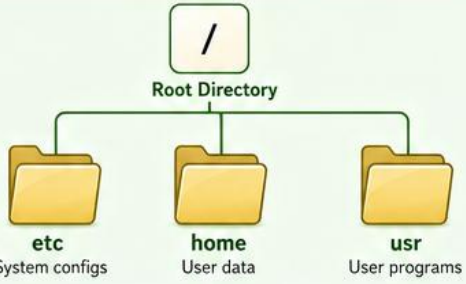
```
nameserver 8.8.8.8
nameserver 1.1.1.1
...
```

Configuration read

2 LOGICAL FILESYSTEM

Provides a logical view of the filesystem as a single directory tree starting from the root (/).

- Hides device and filesystem implementation details
- Manages directories and files in a structured hierarchy



Key Point

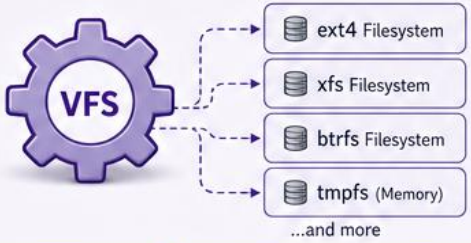
Directories are just special files that contain references to other files or subdirectories.

File operations (open, read, write, etc.)

3 VIRTUAL FILESYSTEM (VFS)

Acts as an abstraction layer between applications and actual filesystems.

- Provides a unified interface for different filesystem types
- Translates generic file operations to specific filesystem operations
- Supports multiple filesystem types simultaneously



Benefits

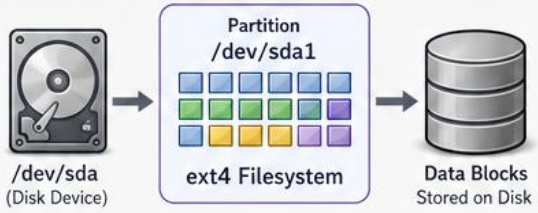
- One interface, many filesystems
- Easier integration and maintenance
- Extensible and flexible design

Block I/O requests (read, write, etc.)

4 PHYSICAL FILESYSTEM

Manages actual data on storage devices.

- Data is stored on partitions
- Uses a specific filesystem implementation (e.g., ext4)
- Handles block allocation, journaling, metadata, and integrity

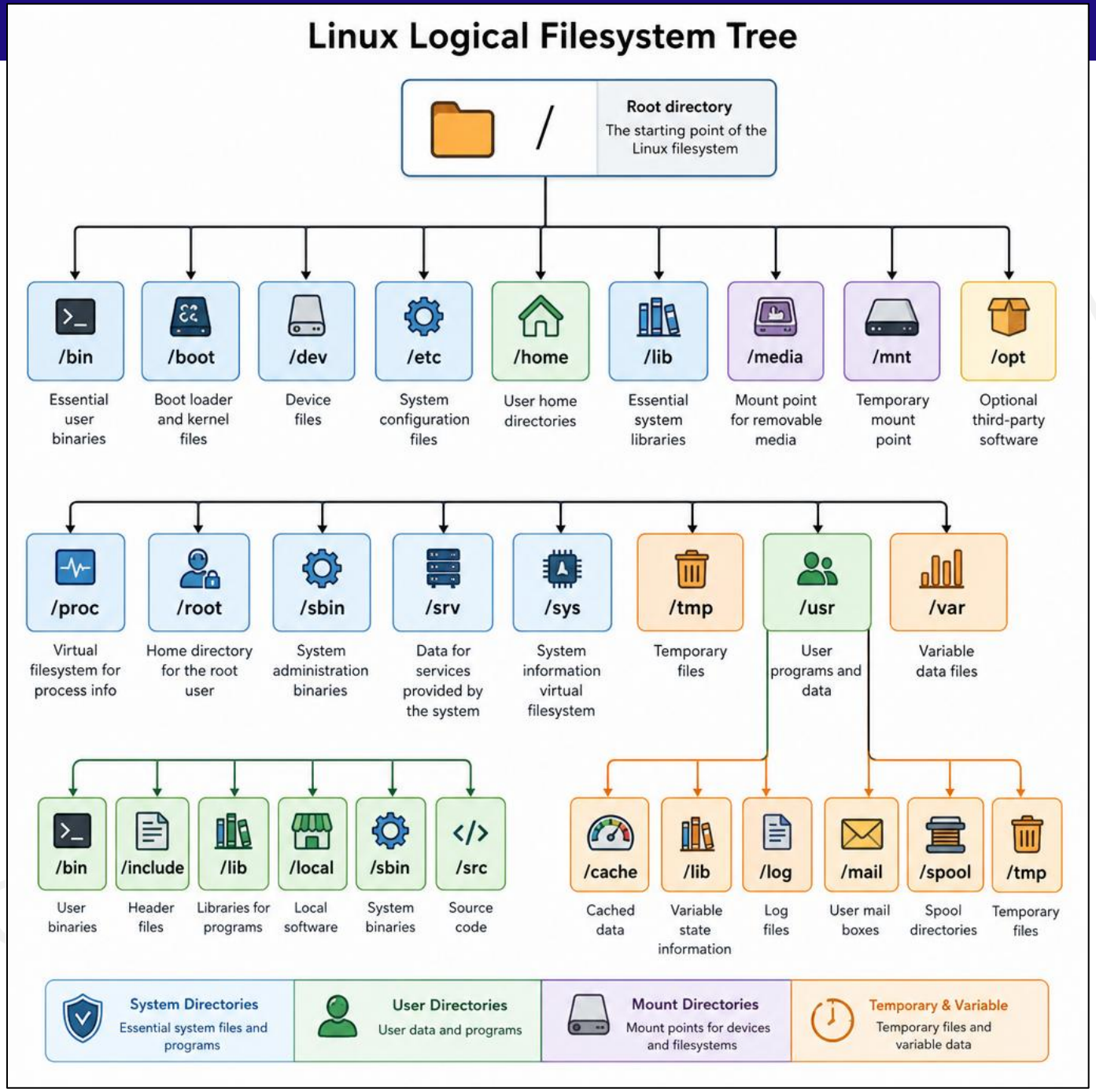


Example

Data written to a file is broken into blocks and stored on the disk within a specific filesystem (e.g., ext4) on a partition.

FLOW OF OPERATIONS 1 Read Config → 2 Logical View → 3 VFS Abstraction → 4 Physical Storage

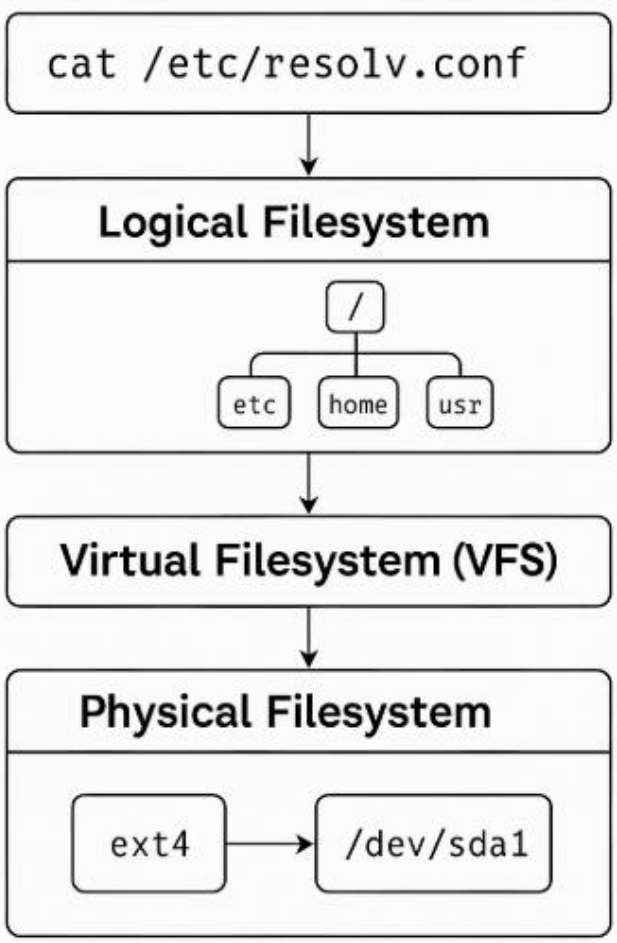
Linux Logical Filesystem Tree



HOW FILE SYSTEM WORKS?

The Linux file system is made up of **3-layers**:

1. The **Logical File System** serves as the **interface** between user applications and the file system, managing operations like opening, reading, and closing files. Directory layout, LVM logical Volumes, Mounted Filesystems
2. **Virtual File System is an abstraction layer inside the Linux kernel.** Provides a unified interface to access all types of filesystems, for example, physical filesystems (ex4,xfs), Network filesystems (NFS) & Pseudo-filesystems (procfs, sysfs, tmpfs). Run cat/proc/cpuinfo which is DATA but looks like a normal file
3. **Physical File System is the actual data on the disk** organized into **blocks**, sectors, **inodes**, superblocks defined by formats like "**ext4**, xfs, btrfs, vfat" created using tools like "**mkfs**". It is what is physically **written to and read from** the storage device.



LS COMMAND

LISTING
#ls
#ls –l /
#ls –ld /
#ls –al /
#ls –il /

```
[root@myserver /]# ls -ld /
dr-xr-xr-x. 22 root root 287 Jul 28 01:23 /
```

- 287bytes = This is the size of the **directory inode**, not what's inside it.
- This size may grow as more files/directories are added to /, but still, it is only the **directory's metadata**, not the sum of its contents.

```
[nitadmin2@nitadmin ~]$ #How can I see my home directories?
[nitadmin2@nitadmin ~]$ ls -al
total 20
drwx------. 2 nitadmin2 nitadmin2   99 Dec 21 15:50 .
drwxr-xr-x. 3 root      root        23 Dec 20 20:49 ..
-rw-------. 1 nitadmin2 nitadmin2 1086 Dec 25 01:23 .bash_history
-rw-r--r--. 1 nitadmin2 nitadmin2   18 Apr 30  2024 .bash_logout
-rw-r--r--. 1 nitadmin2 nitadmin2  141 Apr 30  2024 .bash_profile
-rw-r--r--. 1 nitadmin2 nitadmin2  492 Apr 30  2024 .bashrc
-rw-------. 1 nitadmin2 nitadmin2   20 Dec 21 15:50 .lesshst
```

ls -ld / shows the metadata of the '**/**' directory

This is different from **ls -l /**, which lists the metadata of the *contents inside* the /boot directory

Furthermore, **ls –il /** gives the inode numbers

What is Meta Data?

- **File type**
- **File size**
- **File permission**
- **File owner and group**
- **Timestamp**

BIG DATA

Every file in Linux has **two parts** — the **data** (what's inside the file) and the **metadata** (information *about* the file).

- The **data** is what we create and edit (text, images, code, etc.).
- The **metadata** is stored in a special structure called an **inode**.

Inodes are the backbone of Linux filesystems. **They store metadata** — the information that helps the Operating System (OS) manage files efficiently. Without inodes, Linux wouldn't know where your files are or who owns them

An inode is like a record card in a library — it doesn't hold the book's content, but it tells you everything about the book.

Each inode stores:

- File type (regular file, directory, link, etc.)
- File size
- Permissions (who can read/write/execute)
- Owner and group
- Timestamps (created, modified, accessed)

What is Meta Data?

- **File type**
- **File size**
- **File permission**
- **File owner and group**
- **Timestamp**

How Linux Uses Inode Numbers to Locate Files and Show Metadata (`ls -l`)

Inodes store information about files (metadata), not the file names.

1 You run a command

```
$ ls -l
```

2 Linux reads the directory

The OS opens the current directory (i.e., reads its inode).

3 Directory lists names and inode numbers

Directory Inode (e.g., 3470001)

Name	Inode No.
file1.txt	3470043
file2.log	3470023
docs	3470031
image.png	3470065

What's happening?

The directory inode contains a list of filenames and the inode numbers assigned to them.

4 For each name, Linux uses the inode number

The OS looks up each inode number in the inode table.

5 Linux reads the inode to get metadata

Inode Table (on disk)

Inode No.	Type	Permissions	Owner/Group	Size (bytes)	Timestamps (atime, mtime, ctime)	Data Block Pointers
3470043	-	-rw-r--r--	user/group	1234	May 24 10:15	...
3470023	-	-rw-r--r--	user/group	2048	May 23 09:40	...
3470031	d	drwxr-xr-x	user/group	4096	May 20 08:30	...
3470065	-	-rw-r--r--	user/group	51200	May 22 14:50	...

Inodes store METADATA

File type, permissions, owner, size, timestamps, and pointers to data blocks (not the name).

6 Linux collects the metadata

Now Linux has all the information about each file from their inodes.

7 Format the output

Linux formats the metadata into a long listing format (`ls -l`).

8 Display the result

You see the metadata for each file and directory.

Key Point

Linux uses inode numbers as unique IDs to find and read metadata. That's how commands like `ls -l` show you detailed information about files.

9 Example Output of `ls -l`

```
$ ls -l
total 16
-rw-r--r--  1 user group    1234 May 24 10:15 file1.txt
-rw-r--r--  1 user group    2048 May 23 09:40 file2.log
drwxr-xr-x  2 user group    4096 May 20 08:30 docs
-rw-r--r--  1 user group   51200 May 22 14:50 image.png
```

Permissions | Links | Owner | Group | Size | Timestamp | Filename

Command Quick Guide

`ls -l`	Show long listing (metadata)
`ls -li`	Show inode number + long listing
`df -i`	Show inode usage on filesystem

DAC – DISCRETIONARY ACCESS CONTROL

Permissions

User — read / execute

Group — read / execute

All — read / execute

We will learn later….

[root@myserver ~]# stat /boot

3

dr-xr-xr-x. 5 root root 4096 Jul 13 01:08 /boot

- File Type
- Owner Permission
- Group Permissions
- Other Permissions
- Number of Hard Links
- User
- User's Primary Group
- File Size in bytes
- Modification Date
- File Name

1

Root

"/"

2

Step 1: Let us open the "/boot" directory:

[root@myserver ~]# **ls -ld /boot**

INODES

When a filesystem is created a set amount of **Inodes are created.**
INODE is data structure that keeps track of all the information about a file.
You store your information in a file, and the operating system stores the information about a file in an inode (sometimes called an inode number).
Information about files(data) are sometimes called metadata. So, you can even say it in another way, **"An inode is metadata of the data."**

#ls -ld

Inode No 3470036	
. (DOT)	3470036
..(DOT DOT)	3470017
Folder 1	3470031
File 1	3470043
File 2	3470023
Folder 2	3470024
File 3	3470065

File and Directory Names

Inode corresponding to those names

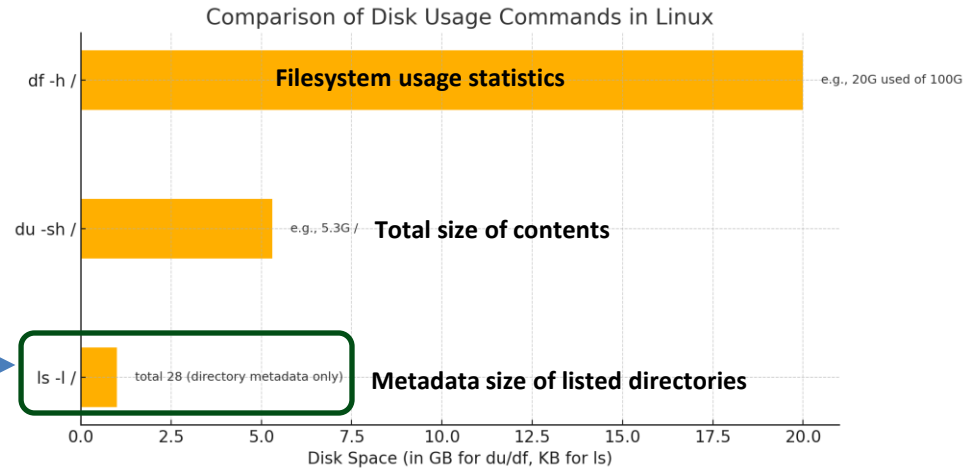
#df - il

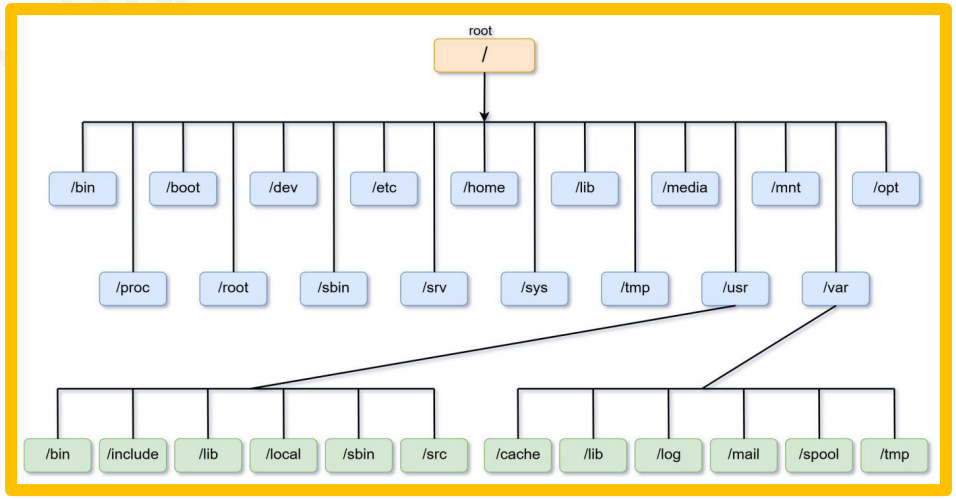
What is Meta Data?

- File type
- File size
- File permission
- File owner and group
- Timestamp

Linux File System (Logically)...

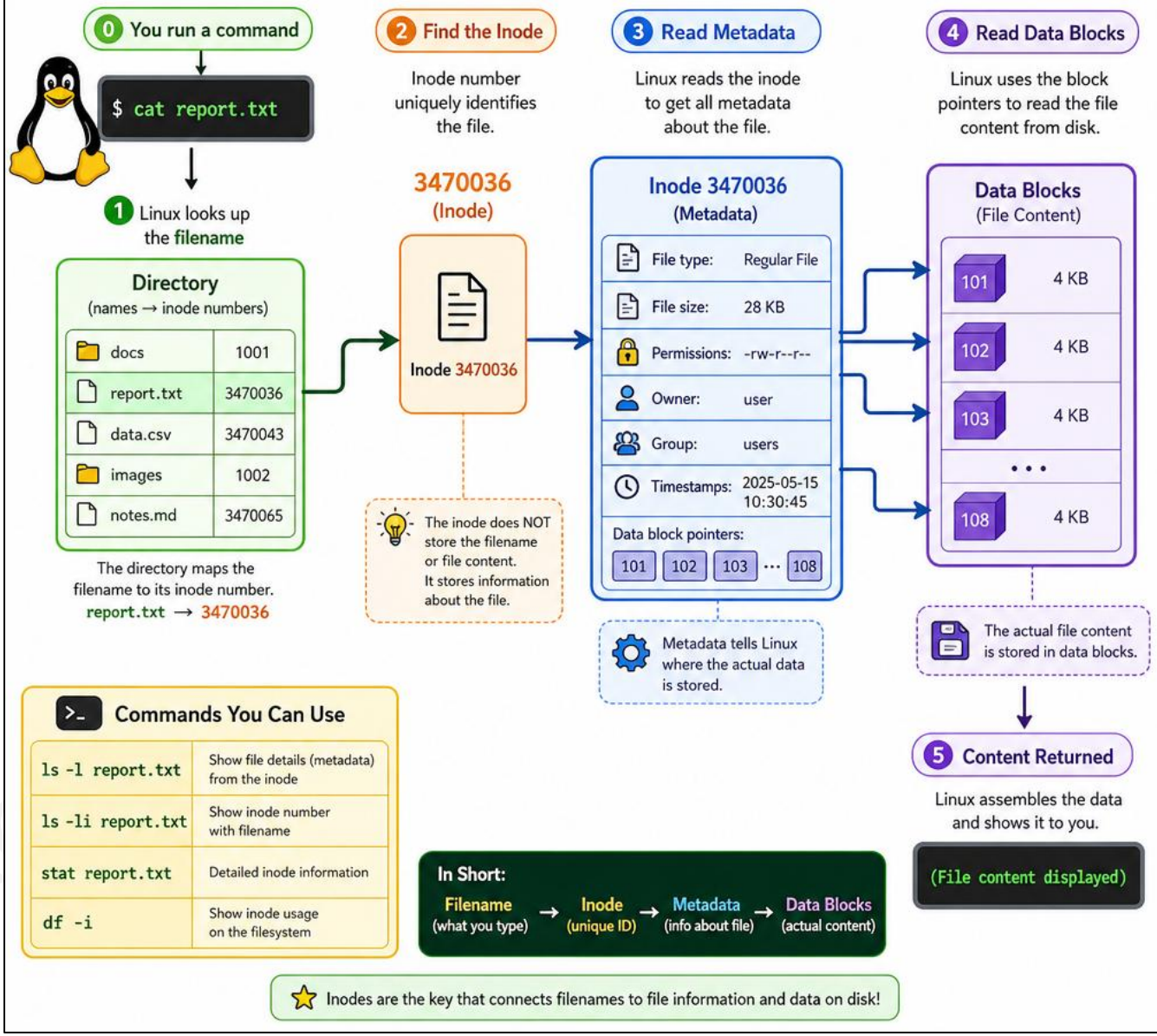
```
[root@myserver ~]# cd /
[root@myserver /]# ls -l
total 28 => 28KB of metadata (not actual data content)
dr-xr-xr-x.   2 root root    6 Nov  2  2024 afs
lrwxrwxrwx.   1 root root    7 Nov  2  2024 bin -> usr/bin
dr-xr-xr-x.   5 root root 4096 Aug  2 02:41 boot
drwxr-xr-x.  21 root root 3380 Aug  2 02:37 dev
drwxr-xr-x. 134 root root 8192 Aug  2 02:50 etc
drwxr-xr-x.   7 root root   77 Jul 27 12:53 home
lrwxrwxrwx.   1 root root    7 Nov  2  2024 lib -> usr/lib
lrwxrwxrwx.   1 root root    9 Nov  2  2024 lib64 -> usr/lib64
drwxr-xr-x.   2 root root    6 Nov  2  2024 media
drwxr-xr-x.   2 root root    0 Jul 28 01:23 misc
drwxr-xr-x.   2 root root    6 Nov  2  2024 mnt
drwxr-xr-x.   2 root root    0 Jul 28 01:23 net
drwxr-xr-x.   2 root root    0 Jul 28 01:23 netdir
drwxr-xr-x.   3 root root   22 Jul 19 20:09 netdir1
drwxr-xr-x.   4 root root   36 Jul 14 18:25 opt
dr-xr-xr-x. 183 root root    0 Jul 28 01:23 proc
dr-xr-x---.   4 root root 4096 Aug  2 02:51 root
drwxr-xr-x.  44 root root 1280 Aug  2 02:48 run
lrwxrwxrwx.   1 root root    8 Nov  2  2024 sbin -> usr/sbin
drwxr-xr-x.   2 root root    6 Nov  2  2024 srv
dr-xr-xr-x.  13 root root    0 Jul 28 01:23 sys
drwxrwxrwt.  11 root root 4096 Aug  2 09:13 tmp
drwxr-xr-x.  12 root root  144 Jul 13 00:28 usr
drwxr-xr-x.  21 root root 4096 Aug  2 02:32 var
```





How Linux Resolves a File Using Inode Numbers

Filename → Inode → Metadata → Data Blocks



FS: SNAPSHOT

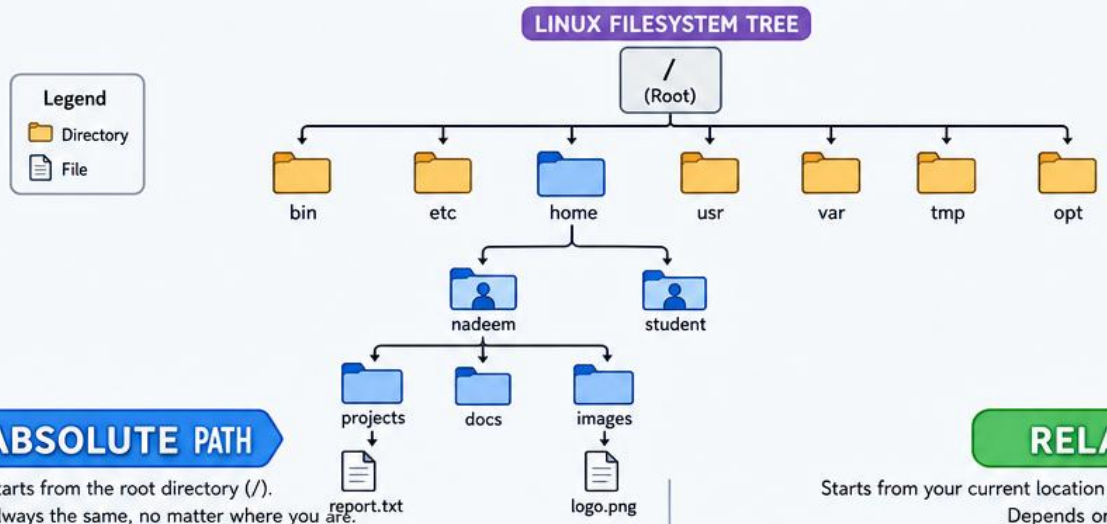
The **Linux filesystem** is structured as a hierarchical tree starting from the topmost directory called the **root (/) directory**. All other directories in Linux can be accessed from the root directory as they are “branches”.

```
Root (/)
|
├── Boot Sector: `Bootloader`
|
├── User Data: `home`
|   ├── user1
|   └── user2
|
├── System Essentials: `bin`, `sbin`, `lib`
|
├── Devices: `dev`
|
├── Configuration: `etc`
|
├── Logs and Cache: `var`
|
├── Temporary Data: `tmp`
|
└── Optional Software: `opt`
```

```
/ (Root)
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

LINUX PATHS: ABSOLUTE vs RELATIVE

Two ways to find a file or directory in Linux

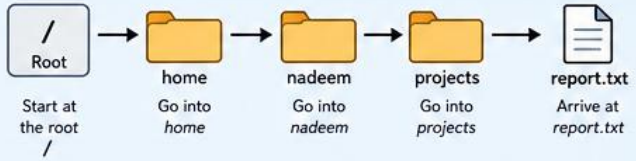


ABSOLUTE PATH

Starts from the root directory (/).
Always the same, no matter where you are.

Example

/home/nadeem/projects/report.txt



Think of it like a full address:
Country > City > Street > House

RELATIVE PATH

Starts from your current location (working directory).
Depends on where you are.

Example Current directory (pwd): **/home/nadeem/projects**

Path to logo.png using relative path:

../images/logo.png

Think of it like directions from where you are:
You are here > Go up > Into images > logo.png

QUICK SUMMARY

- Use **ABSOLUTE PATHS** in scripts, configs, and when you need an exact location.
- Use **RELATIVE PATHS** in daily navigation — shorter and location-friendly.

Handy Commands

```
pwd     # show current directory
ls      # list files and folders
cd ..   # go up one level
```

LINUX FILESYSTEM STRUCTURE

The **Linux filesystem** is structured as a hierarchical tree starting from the topmost directory called the **root (/) directory**. All other directories in Linux can be accessed from the root directory as they are “branches”.

Absolute Path

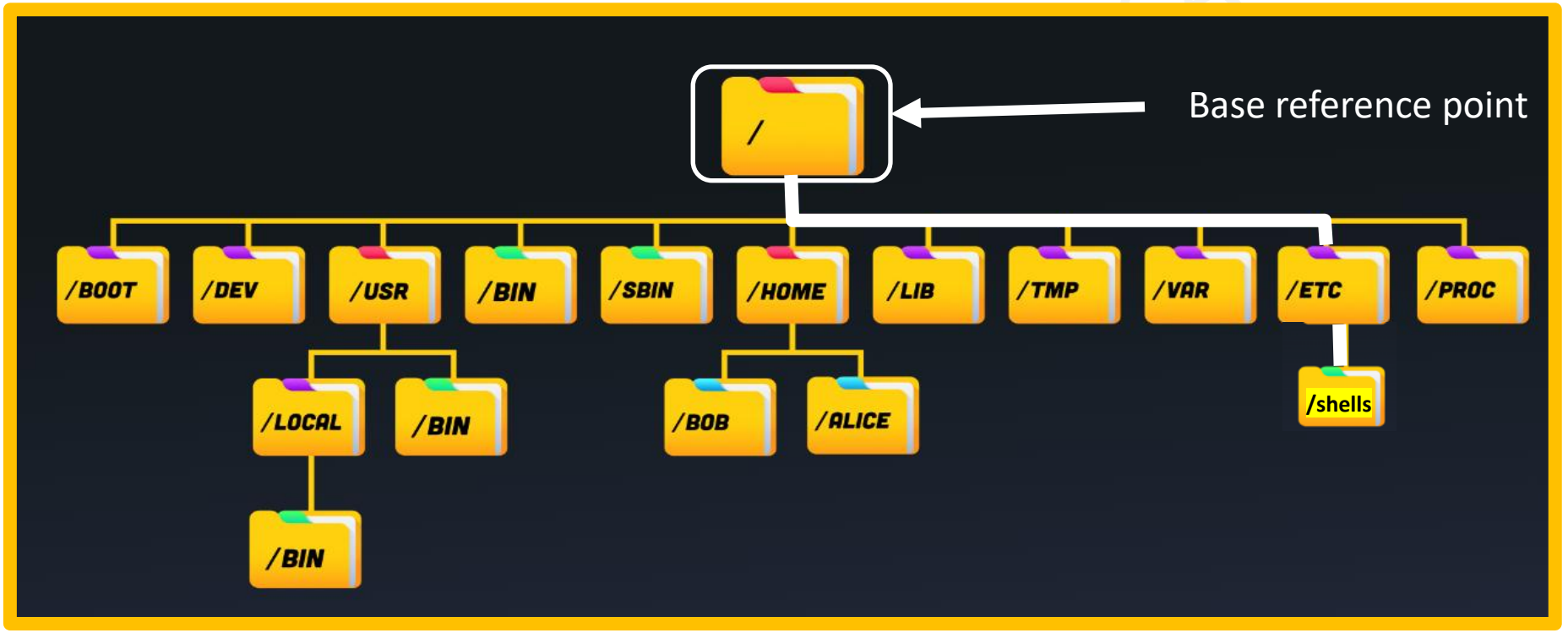
An **absolute path** is the complete path to a file or directory starting from the root (/) directory. It always begins with / and specifies the full location of the file or directory in the filesystem hierarchy, regardless of the current working directory.

ABSOLUTE PATH

An **absolute path** is the complete path to a file or directory starting from the root (/) directory. It **always begins with "/"** and specifies the full location of the file or directory in the filesystem hierarchy, regardless of the current working directory.

Example: "/" – Leading when in beginning and "/" - trailing

[nitadmin2**]$cat /etc/shells**

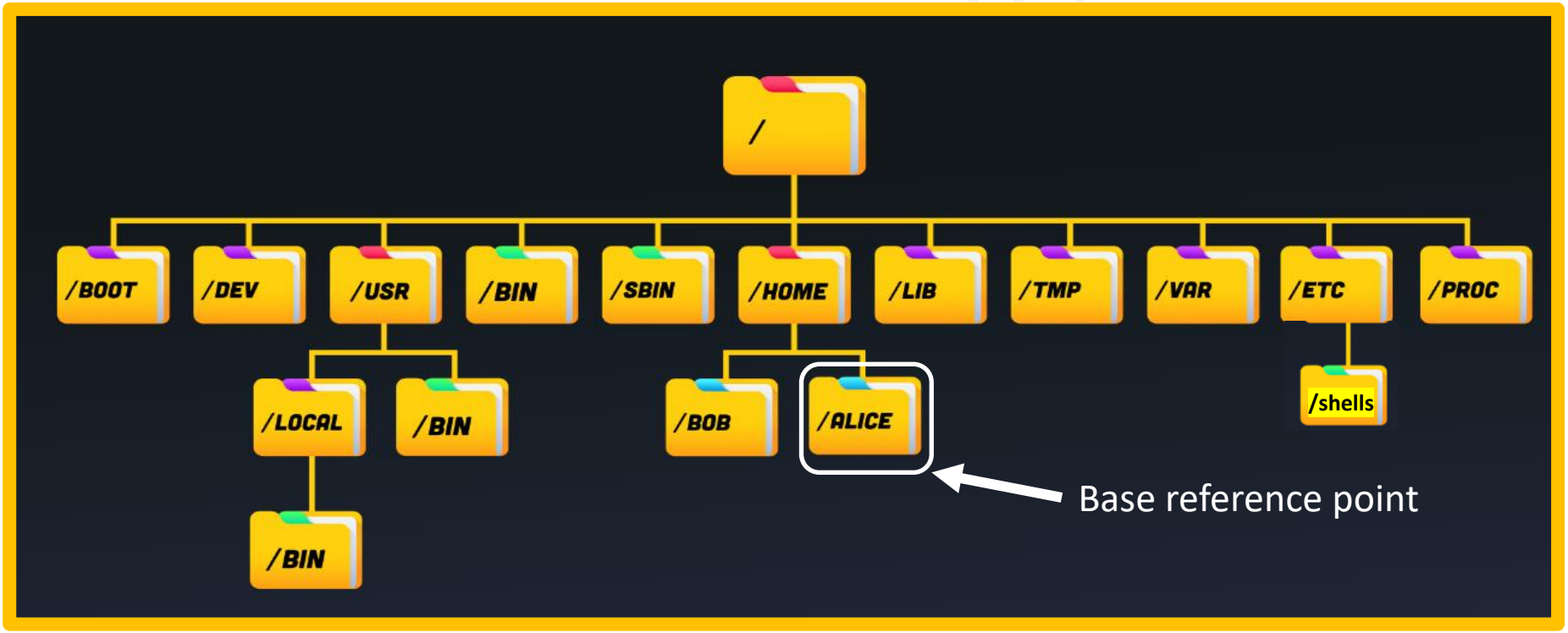


RELATIVE PATH

A Relative path specifies the location of a file or directory relative to the current working directory **(pwd)**. It **does not start with "/"** and instead uses the current directory as the **base reference point**.
Example: User is in "Home" directory and wants to navigate within "Home"

```
[root@myserver alice]# pwd
/home/alice
```

```
[root@myserver alice]# ll
total 4
drwxr-xr-x. 2 root root  6 Aug  2 12:07 dir1
-rw-r--r--. 1 root root 12 Aug  2 12:09 file1
[root@myserver alice]# cd dir1
[root@myserver dir1]#
```



Password Recovery – RHCSA EXAM QUESTION 1

- Crack Password of node2 Machine, and set password to **redhat**

/boot

1. Understand the booting process of Linux
2. How to set default target, used when booting and boot a system to a non-default-target
3. Login to a system and change root password
4. How to manually Repair the filesystem configuration or tackle corruption issue that stop the boot process.
5. Boot, reboot, and shut down a system **normally**

/BOOT

[root@myserver ~]# **ls -l /boot**

total 184536 <= File Size

```
[root@myserver kernel]# ls -l /boot
total 184536
-rw-r--r--. 1 root root   226284 Nov 15  2024 config-5.14.0-503.14.1.el9_5.x86_64
drwxr-xr-x. 3 root root       17 Jul 13 00:28 efi
drwx------. 5 root root       97 Jul 27 17:06 grub2
-rw-------. 1 root root 79130466 Jul 13 00:36 initramfs-0-rescue-4de79e0ace1b4500a934ade7bb133ed1.img
-rw-------. 1 root root 37206781 Jul 13 00:44 initramfs-5.14.0-503.14.1.el9_5.x86_64.img
-rw-------. 1 root root 34598400 Jul 13 01:08 initramfs-5.14.0-503.14.1.el9_5.x86_64kdump.img
drwxr-xr-x. 3 root root       21 Jul 13 00:30 loader
lrwxrwxrwx. 1 root root       52 Jul 13 00:31 symvers-5.14.0-503.14.1.el9_5.x86_64.gz -> /lib/modules/5.14.0-503.14.1.el
9_5.x86_64/symvers.gz
-rw-------. 1 root root  8876141 Nov 15  2024 System.map-5.14.0-503.14.1.el9_5.x86_64
-rwxr-xr-x. 1 root root 14457672 Jul 13 00:33 vmlinuz-0-rescue-4de79e0ace1b4500a934ade7bb133ed1
-rwxr-xr-x. 1 root root 14457672 Nov 15  2024 vmlinuz-5.14.0-503.14.1.el9_5.x86_64
[root@myserver kernel]# ls -ld /boot
dr-xr-xr-x. 5 root root 4096 Jul 13 01:08 /boot
```

Out of all these files, there are four that stand out:

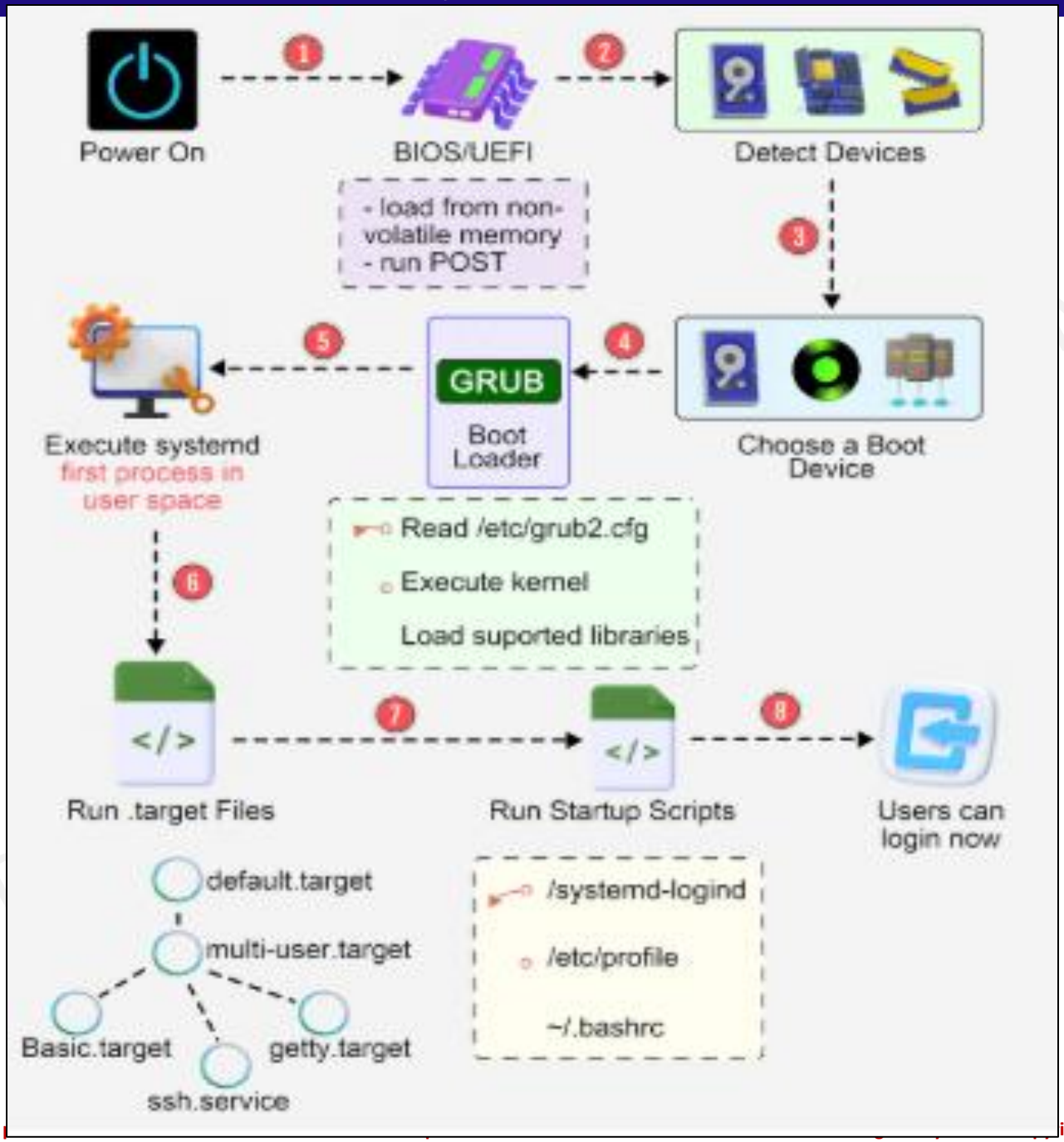
1. The **Configuration** file
2. The **Initial RAM** Disk file
3. The **System Map** file
4. The **Linux Kernel** file

[root@myserver ~]# **tree -d /boot**

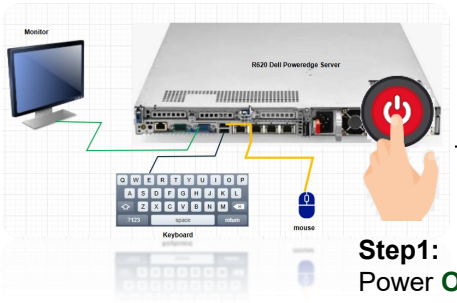
/boot

```
├── efi  (EFI partion exists, but does not imply UEFI boot
│   └── EFI
│       └── rocky
├── grub2
│   ├── fonts
│   ├── i386-pc
│   └── locale
└── loader
    └── entries
```

9 directories



CONTROLLING THE BOOT PROCESS



Step1: Power **ON**

BIOS

Step2: **Basic input/output**

HDD /dev/sda

Step3: **Search for bootable Device**

Fun activity: View MBR and Partition Table!

dd if=/dev/sda bs=512 count=1 | hexdump -C

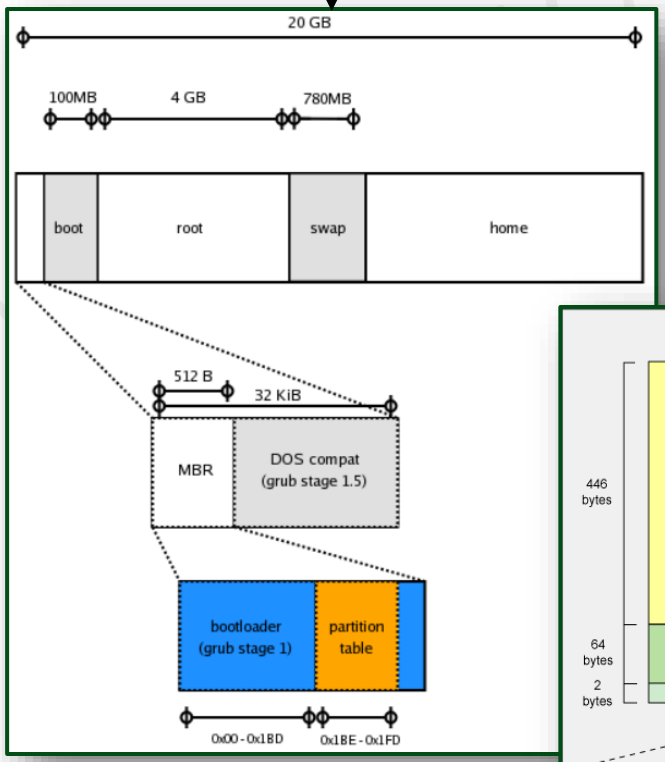
MBR

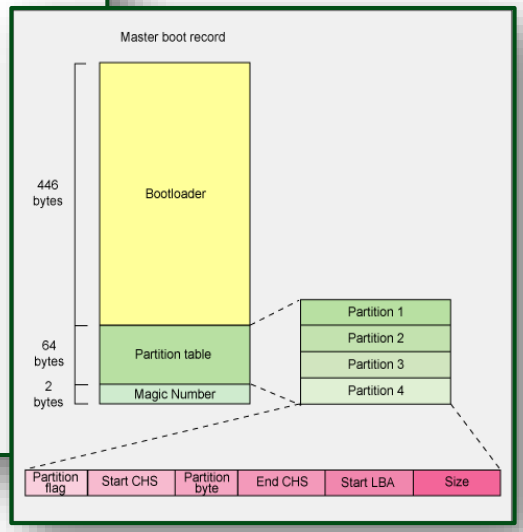
Grub2

[root@myserver boot]# fdisk -l

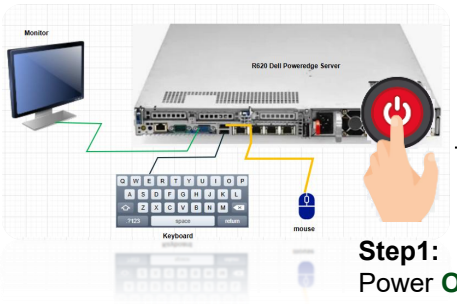
Disk /dev/sda: 20 GiB, 21474836480 bytes, 41943040 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0xb25cfa22

Device	Boot	Start	End	Sectors	Size	Id	Type
/dev/sda1	*	2048	2099199	2097152	1G	83	Linux
/dev/sda2		2099200	41943039	39843840	19G	8e	Linux LVM





CONTROLLING THE BOOT PROCESS



Step1: Power **ON**

BIOS

Step2: **Basic input/output**

HDD /dev/sda

Step3: **Search for bootable Device**

MBR

Grub2

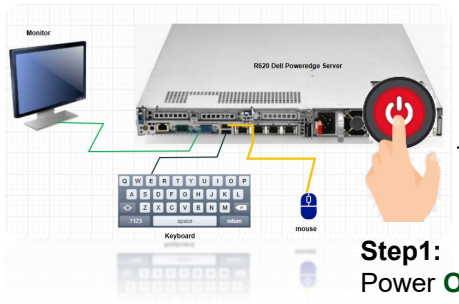
Simply loads the the kernel into the memory

GRUB stands for Grand Unified Bootloader

- GRUB2 (Grand Unified Bootloader) Version 2.
- GRUB2 loads its configuration from the file /boot/grub2/grub.cfg file and displays a menu (splash screen) where you can select which kernel to boot.
- It loads the vmlinuz kernel Image (/boot/vmlinuz-4.18.0-80.el8.x86_64) and extract the content of Initramfs image (initramfs-4.18.0-80.el8.x86_64.img)

Random Access Memory (RAM)

CONTROLLING THE BOOT PROCESS



Step1: Power **ON**

BIOS

Step2: **Basic input/output**

HDD /dev/sda

Step3: **Search for bootable Device**

MBR

Grub2

Kernel, initramfs

Kernel Initializes all hardware

Executes /sbin/init

Systemd executes initrd.target

Mounting root fs on /sysroot directory

Root fs is switched

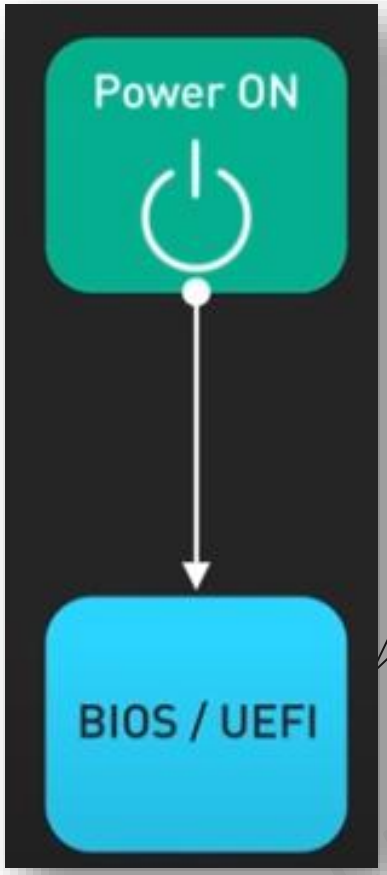
Systemd looks for default.target

Systemd looks for default.target

/etc/system/system/default.target
/usr/lib/system/system/multi-user.target

BOOT PROCESS EXPLAINED

[-d /sys/firmware/efi] && echo "UEFI mode" || echo "Legacy BIOS mode"



BIOS	UEFI
- BIOS is tied to the Master Boot Record(MBR) system, which limits disk size to 2TB - Slower boot time - Less secure boot	- UEFI, on the other hand, uses the GUID Partition Table (GPT), removing these size constraints and offering a more flexible and modern solution - Faster boot time - Secure boot

[nitadmin1@nit firmware]$ **ls –l /sys/firmware/efi**

- If this directory exists, your system is using UEFI.
- If it does not exist, your system is using BIOS.

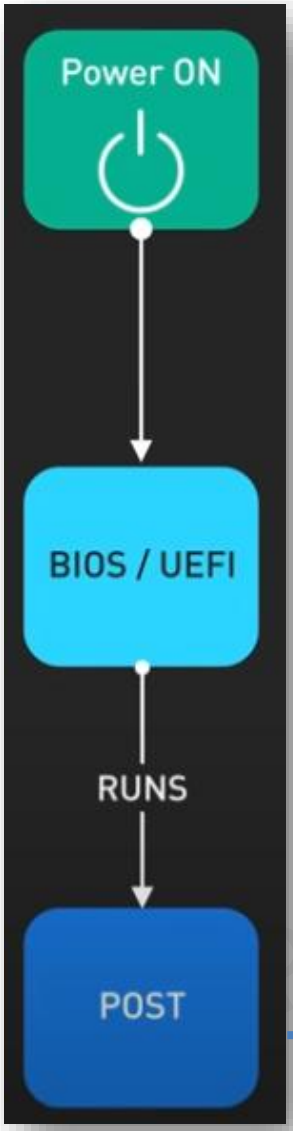
BIOS (Basic Input/Output System)

- **Older firmware standard** used in PCs before UEFI.
- Uses **MBR (Master Boot Record)** for booting, which has a 2TB disk size limit and supports up to **4 primary partitions**.
- **Runs in 16-bit mode**, meaning it has slower boot times and limited features.
- Uses a **text-based interface** for configuration.
- Relies on **Legacy Boot** mode, which loads the bootloader from a specific disk sector.

UEFI (Unified Extensible Firmware Interface)

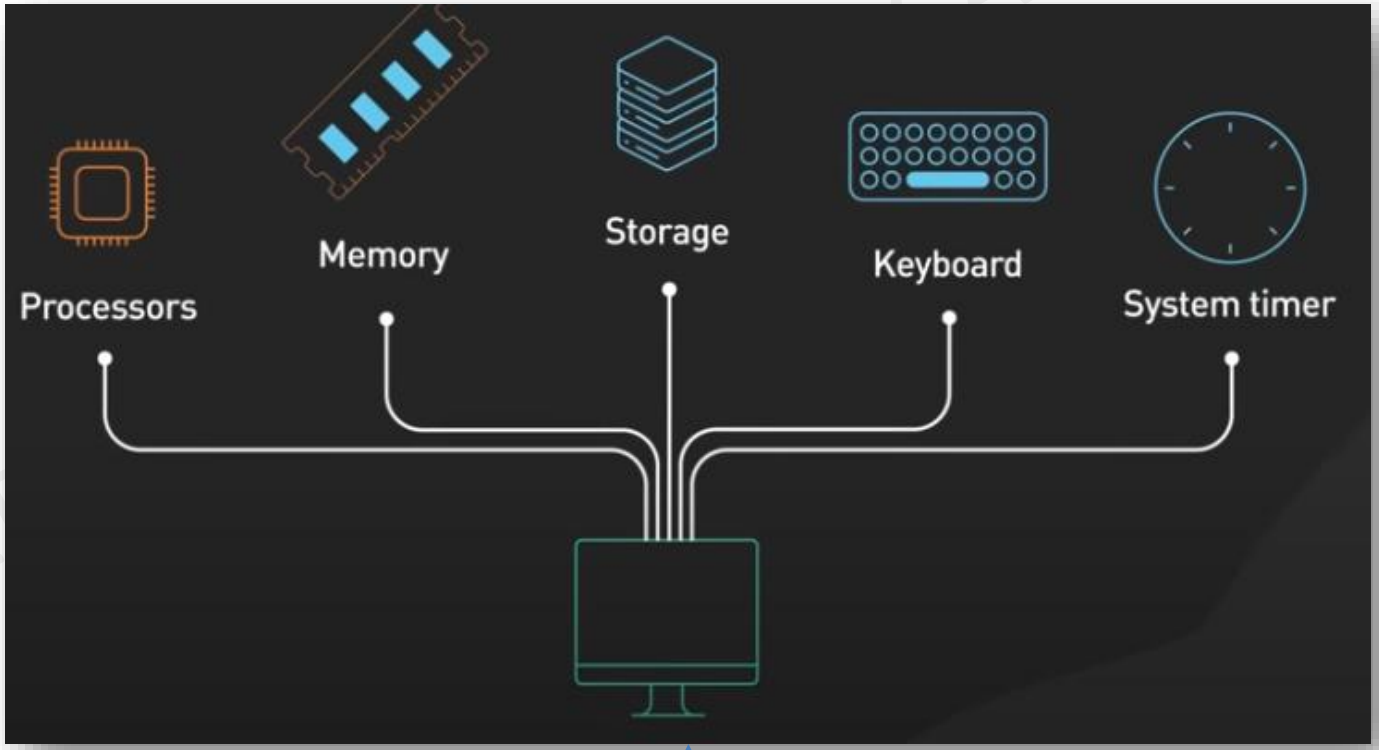
- **Modern replacement for BIOS** with more features and better hardware compatibility.
- Uses **GPT (GUID Partition Table)**, which supports **disks larger than 2TB** and allows **unlimited partitions (typically up to 128)**.
- **Runs in 32-bit or 64-bit mode**, improving speed and efficiency.
- Provides a **graphical interface** with mouse support.
- Includes features like **Secure Boot**, which prevents unauthorized software from running at startup.

BOOT PROCESS - POST



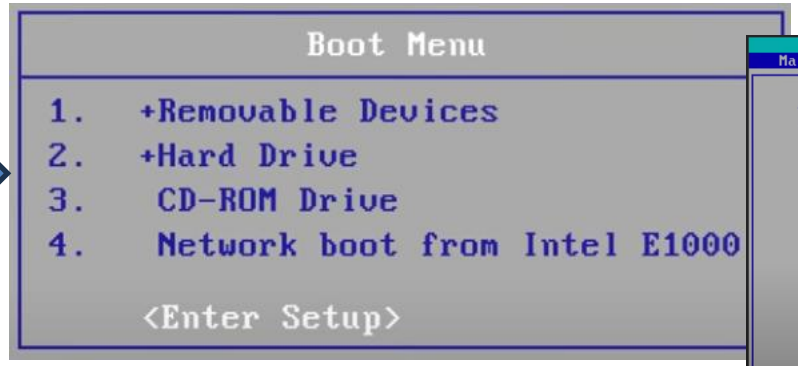
Power **O**n **S**elf **T**est **(POST)**

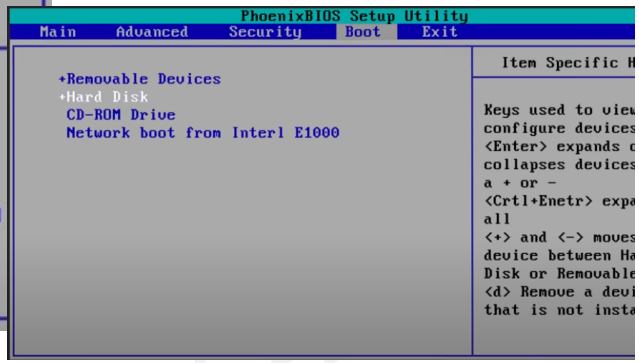
- **CPU Check** – Verifies if the processor is working.
- **Memory Test** – Checks if RAM is detected and functioning.
- **Storage Devices** – Ensures hard drives and SSDs are present.
- **Graphics Card Test** – Checks if the GPU is operational.
- **Input Devices** – Ensures the keyboard and mouse are detected.
- **Other Hardware Checks** – Includes network cards, sound cards, and other peripherals.



BOOT PROCESS – PARTITIONS

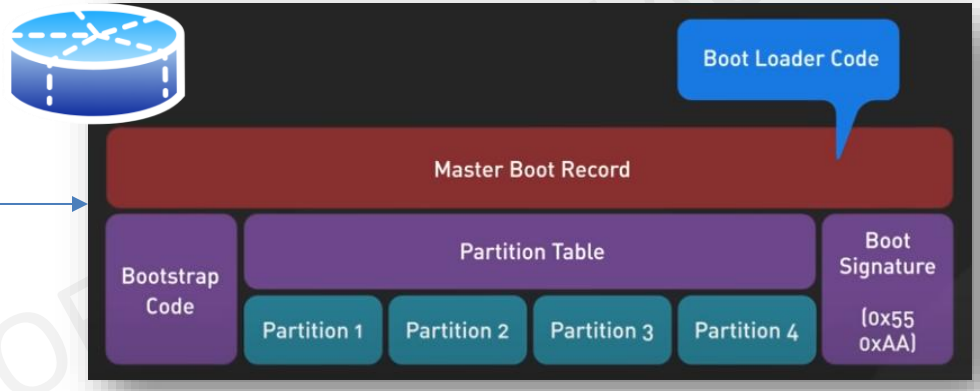
BIOS / UEFI



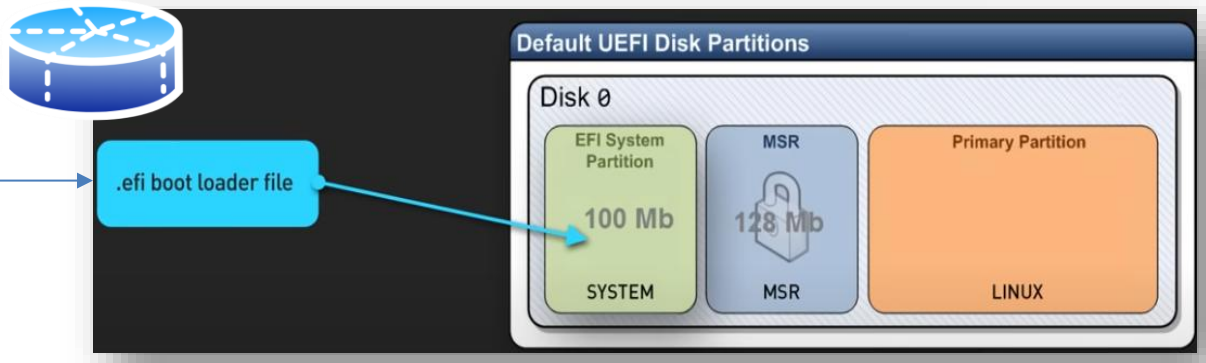


Partition

BIOS



UEFI



BOOT PROCESS – BOOT LOADER(S) -> KERNEL

BIOS / UEFI

MBR / efi

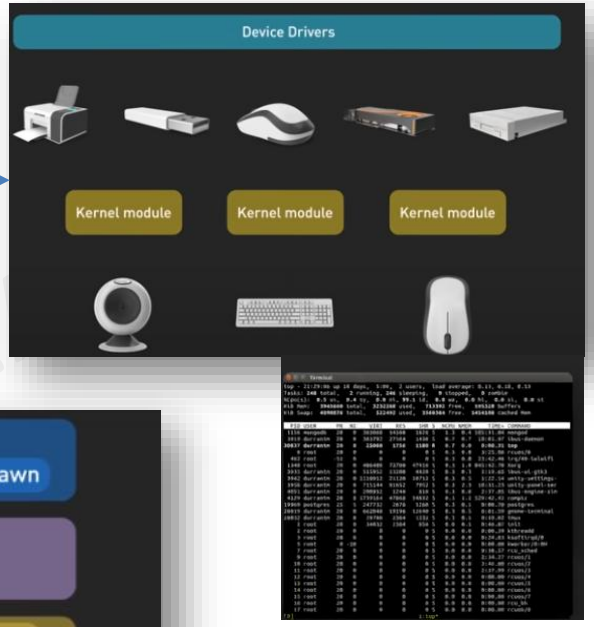
BOOTLOADER

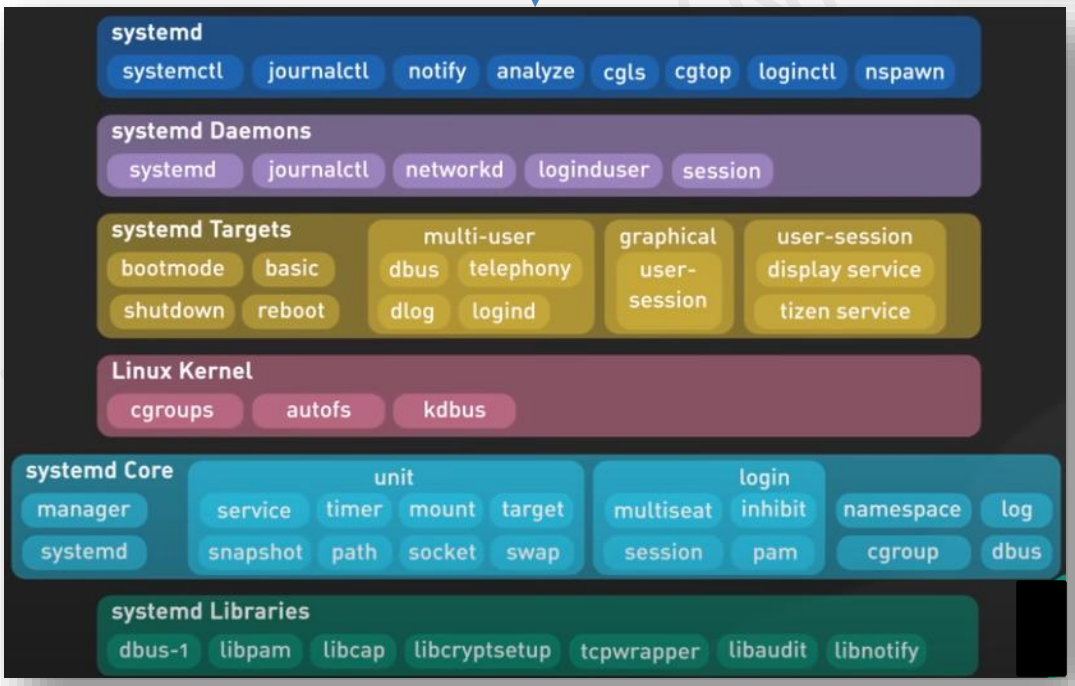
GRUB2 / ~~LILO~~

The key jobs for the boot loader are:

1. Locate the operating system kernel on the disk
2. Load the kernel into the computer's memory
3. Start running the kernel code

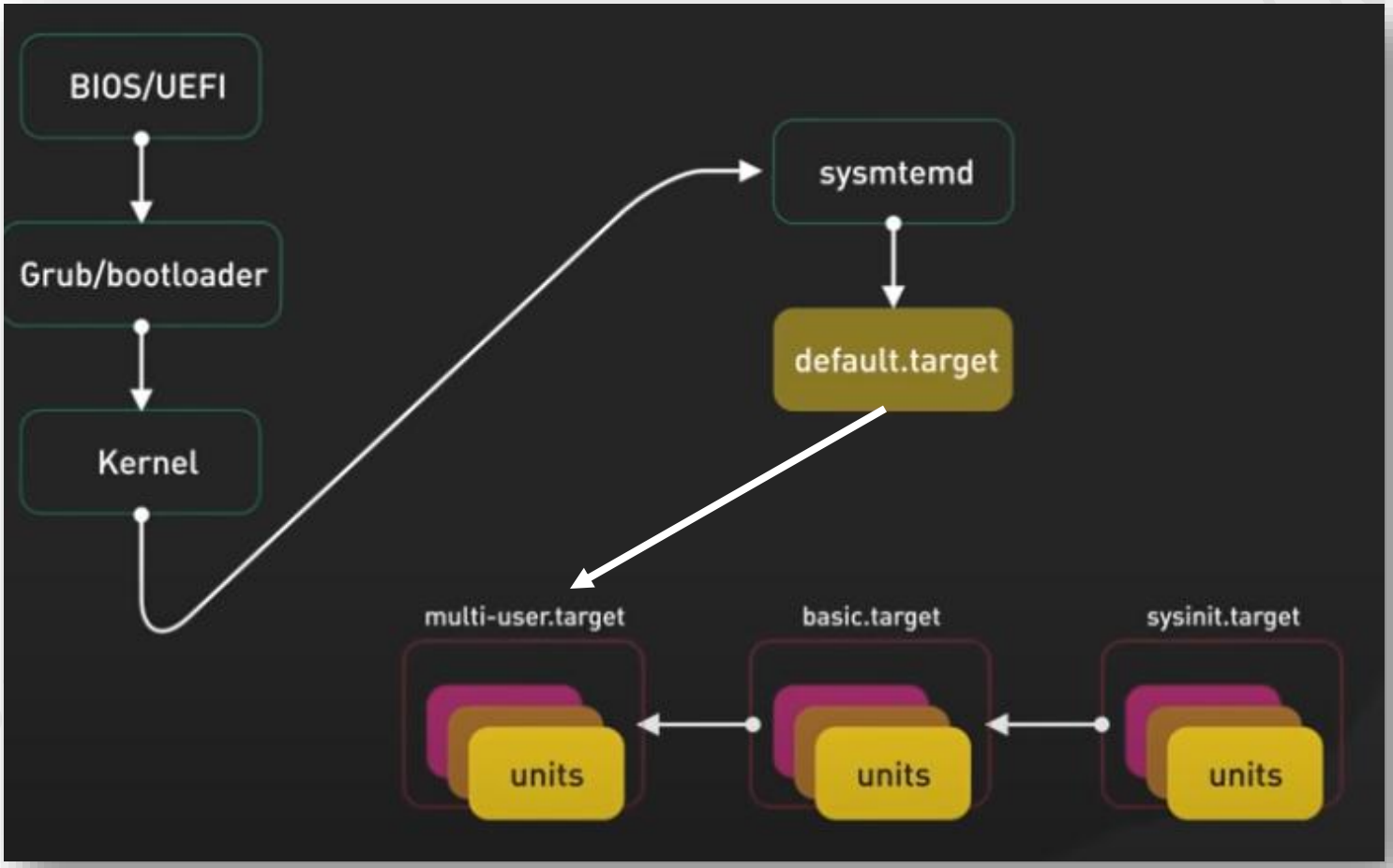
Kernel





BOOT PROCESS – TARGET

In **systemd**, a ***target*** is a group of services that define the state of the system. Targets replace traditional SysV runlevels and control what services and resources are available at any given time



TARGET – LINUX COMMANDS

How to check your systemd target
[nitadmin1@nit firmware]$ systemctl get-default
multi-user.target

To set default target permanently
To set a new default boot target:
#systemctl set-default multi-user.target

Switch to a different target in a running system
For example, to switch to **rescue mode**:
#systemctl isolate rescue.target

Run Level	Target Units	Description
0	runlevel0.target , poweroff.target	Shutdown and poweroff the system
1	runlevel1.target , rescue.target	Set up rescue shell
2	runlevel2.target, multi-user.target	Set up non-graphical multi-user system
3	runlevel3.target, multi-user.target	Set up non-graphical multi-user system
4	runlevel4.target, multi-user.target	Set up non-graphical multi-user system
5	runlevel5.target, graphical.target	Set up graphical multi-user system
6	runlevel6.target, reboot.target	Shut down and reboot the system

Here are some frequently used targets you can load with `systemd.unit=<target>` at boot:

Target	Description
`default.target`	The default target the system boots into (usually linked to `graphical.target` or `multi-user.target`).
`rescue.target`	Single-user mode with minimal services, useful for system recovery (like old **runlevel 1**).
`emergency.target`	Similar to `rescue.target`, but loads only the most basic system components.
`multi-user.target`	Command-line multi-user mode, similar to old **runlevel 3**.
`graphical.target`	Multi-user mode with a graphical user interface (GUI), like old **runlevel 5**.
`reboot.target`	Reboots the system immediately.
`poweroff.target`	Powers off the system.
`halt.target`	Stops the system without powering it off.
`suspend.target`	Suspends the system (sleep mode).
`hibernate.target`	Hibernates the system.
`hybrid-sleep.target`	Combination of suspend and hibernate.
`runlevelX.target`	Backward compatibility with SysV (`runlevel1.target` , `runlevel3.target` , etc.).
`network.target`	Ensures the network is available but does not configure it.

Run Level = init {0,1,2,3,4,5,6}

LINUX COMMANDS

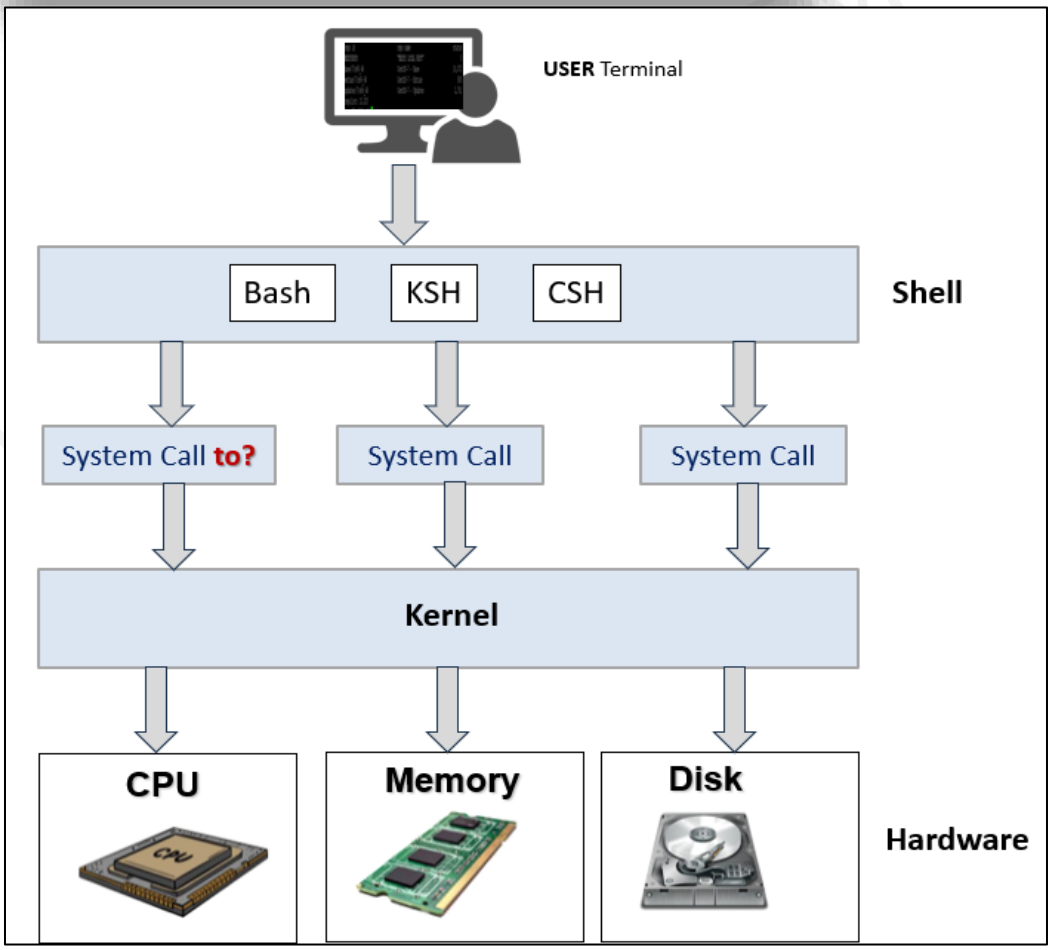
Reboot, Shutdown and Suspending System

Command	Action/Description
systemctl reboot or shutdown -r now	To reboot System
systemctl halt or shutdown --halt	To halt the System
systemctl poweroff or shutdown --poweroff	To power off System
systemctl suspend	To suspend System (no-power mode)
systemctl hibernate	To hibernate System (low-power mode)
shutdown --halt HH:MM	To schedule System halt
shutdown --halt +10	To Schedule System halt after 10 minutes
shutdown --halt +10 “System will go down in 10 minutes”	To schedule System halt with message to users
shutdown -c	To cancel the scheduled shutdown command

/home

1. **Where are you when you login?**
   - Home directory
   - Login directory
   - Contains User personal files, directories and programs.
   - This is where a person lands after logging into the system
2. **Home Directory** is created automatically when a new user is added **with /home**
3. **Users need an environment** that gives them access to the tools and resources. When a user logs in to a system, the user's work environment is determined by the initialization files

```
Root (/)
|
├── User Directory: `home`
|    ├── user1
|    └── user2
```



/dev

```
|
├── Devices: `dev`
```

- **Everything is a file in Linux. Even the devices are viewed as files**
- /dev stands for **"device"**. Reading from /dev/sda is like accessing the hard disk
- **Contains special files that represent hardware devices**
- **Managed dynamically by udev (device manager)**

1 **#ls /dev**

2 **#ls -l /dev/sda1**

```
[root@VM01 etc]# ls -l /dev/sda1
brw-rw----. 1 root disk 8, 1 Jan 15 09:54 /dev/sda1
[root@VM01 etc]# df -h
Filesystem                Size  Used Avail Use% Mounted on
devtmpfs                  4.0M     0  4.0M   0% /dev
tmpfs                     385M     0  385M   0% /dev/shm
tmpfs                     154M  4.3M  150M   3% /run
/dev/mapper/rl_vbox-root   17G  1.6G   16G  10% /
/dev/sda1                 960M  374M  587M  39% /boot
tmpfs                      77M     0   77M   0% /run/user/1000
```

Types of Device Files

Date: Aug-3rd-2025

Type	Description	Examples
Block Devices	Accessed in blocks (random I/O)	/dev/sda, /dev/sdb1
Character Devices	Accessed as a stream (sequential I/O)	**/dev/tty,** /dev/random

1. When accessing Data in Blocks (Block Devices)
Real-Life Devices:
- Filesystems (like ext4, xfs) use block devices
- Stored under **/dev** as **/dev/sda**, **/dev/xvda**, etc.

Examples:
- Hard drives **(/dev/sda)**
- SSDs
- USB drives

Behavior:
- Data is read or written in **fixed-size chunks**, called **blocks** (typically 512 bytes, 4KB, etc.)
- You **can jump to any block** directly without reading all data before it.

Analogy:
Like reading **pages from a book** — you can go to **page 100** directly without reading pages 1–99.

2. When accessing Data as a stream (Character Devices)
Real-Life Devices:
- Serial consoles, pipes, sensors
- Found as /dev/tty*, /dev/zero, /dev/random

Examples:
- Keyboard (/dev/tty)
- Mouse (/dev/input/mouse0)
- Serial ports (/dev/ttyS0)

Behavior:
- Data is accessed as a **continuous flow** (byte-by-byte or character-by-character).
- You **cannot seek** or skip ahead — you read data **in the order it comes in**.

Analogy:
Like **listening to a radio** — you hear sounds as they come; you can't skip ahead.

Types of TTYs in Your Screenshot:

TTY	Explanation
tty1	A **virtual console** directly on the machine
pts/0	A **pseudo-terminal** slave used for **remote or terminal emulator sessions**, like SSH

```
[root@myserver ~]# w
 01:03:26 up 15 min,  2 users,  load average: 0.03, 0.07, 0.09
USER     TTY      LOGIN@   IDLE   JCPU   PCPU WHAT
root     tty1     00:48   14:30   0.01s  0.01s -bash
root     pts/0    01:01    0.00s  0.05s  0.02s w
```

Command	What it does	Terminal 1	Terminal 2
tty	Shows which terminal (e.g., /dev/pts/0)	**[root@myserver ~]# tty** /dev/pts/0	**[root@myserver ~]# tty** /dev/pts/1
w / who	Shows all logged-in users with TTY	**[root@myserver ~]# w** 01:46:22 up 57 min, 4 users, load average: 0.07, 0.06, 0.05 USER TTY LOGIN@ IDLE JCPU PCPU WHAT root tty1 00:48 38:06 0.26s 0.26s -bash root pts/0 01:01 5:49 0.05s 0.05s -bash root pts/1 01:12 0.00s 0.30s 0.13s w	
ls /dev/pts	Lists all pseudo terminals	[root@myserver ~]# ls /dev/pts 0 1 2 ptmx	
echo "msg" > /dev/pts/X	Sends a message to another terminal	echo "Hello from pts/0" > /dev/pts/1	

[root@myserver ~]# lsblk

```
[root@myserver ~]# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda           8:0    0   20G  0 disk
├─sda1        8:1    0    1G  0 part /boot
└─sda2        8:2    0   19G  0 part
  ├─rl-root 253:0    0   17G  0 lvm  /
  └─rl-swap 253:1    0    2G  0 lvm  [SWAP]
sr0          11:0    1 1024M  0 rom
```

Disk Commands

- **#df -hT**
- **#lsblk**
- **#fdisk -l**

Device	Description
/dev/sda	First hard disk (HD)
├─ /dev/sda1	First **partition** on first disk => 1GB mounted as /boot
├─ /dev/sda2	Second partition on first disk =>19 GB **not directly mounted**, but subdivided
│ ├─ rl-root	Logical volume (holds **/**) in a volume group called “rl”
│ ├─ rl-swap	Used as “swap” space
/dev/null	Discards all input
/dev/tty	Terminal (Console) device
/dev/pts/*	Pseudo terminals

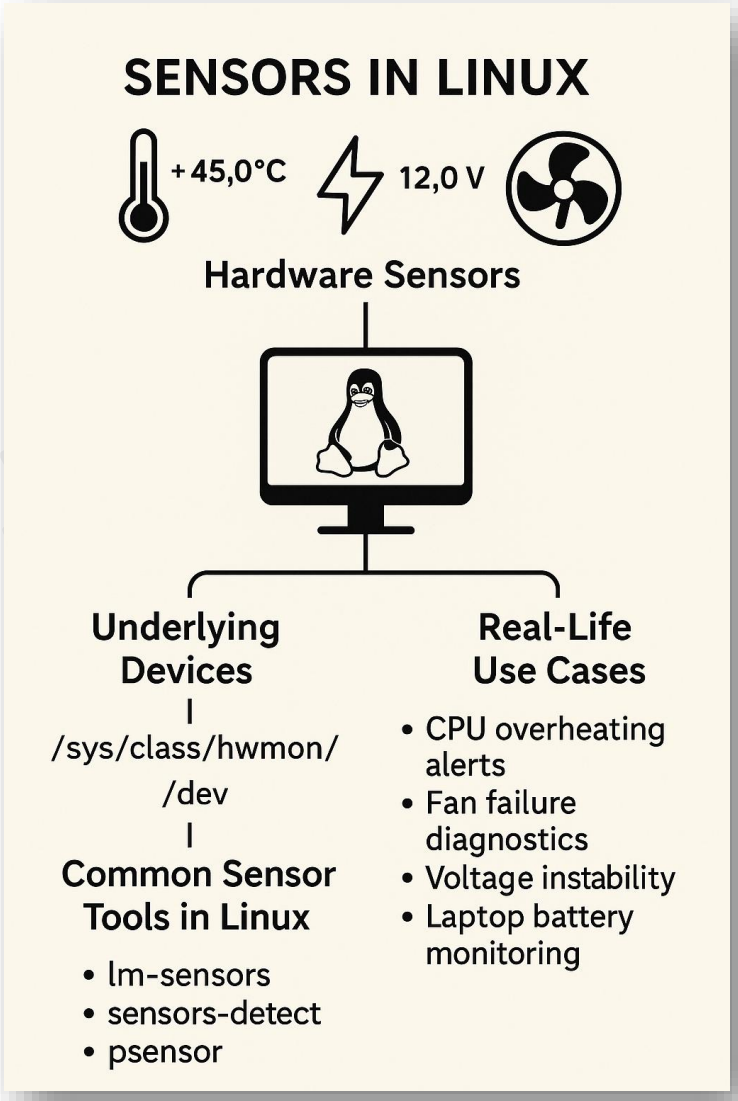
Fun with Sensors in Data Center Environment!!!

VMs on Virtual Box do not give real outputs. This is an important command used in data center equipment audit.

Step 1: Install lm-sensors
dnf install lm_sensors
Or yum install lm_sensors -y
For Debian / Ubuntu => apt install lm-sensors

Step 2: Detect sensors
sensors-detect
Answer **"YES"** to all prompts

Step 3: Check CPU temperature
sensors



/etc

#ls /etc

Configuration Hub

The **/etc** directory is one of the most **critical system directories**. It holds **system-wide configuration files**

- If you're **troubleshooting or configuring** the system, /etc is where you'll spend most of your time.
- Stores **configuration files** for the system and installed services.
- Affects how the system behaves on startup and runtime.

System-Wide Settings

- Configurations in /etc apply to **all users**, unlike ~/.config (per user).

Contains Text Files

•Most config files are **readable text files** and can be edited with **vi editor**

Critical Subdirectories

- /etc/network/ – Network configuration (Debian)
- /etc/sysconfig/ – RHEL/CentOS network, firewall, and service configs
- /etc/ssh/ – SSH server/client settings
- /etc/systemd/ – systemd unit files
- /etc/cron.d/, /etc/crontab – Scheduled tasks
- /etc/yum.repos.d/ – YUM repository configurations

User and Authentication Files

- /etc/passwd – User account information
- /etc/shadow – Encrypted passwords
- /etc/group – Group definitions
- /etc/sudoers – sudo access control (edit with **visudo**).

Boot-Time Configuration

Contains files that control **startup behavior**, services, and kernel modules.

Back It Up!

Always **backup /etc** before making changes. A mistake can break the system

USERS - RHCSA EXAM QUESTION

/var

#ls /var

Configuration Hub
The /var directory stands for **"variable"** – it stores files that **change frequently** during system operation.

Dynamic Data Storage
- Holds **runtime data** that changes, such as logs, emails, and caches.
- Unlike /etc (static configs), /var content **grows over time**.
- \

Important Subdirectories
- /var/log/ – **System logs** (e.g., /var/log/messages, /var/log/secure)
- /var/spool/ – Jobs waiting (**cronjobs** included) to be processed (e.g., print or mail queue)
- /var/cache/ – Cached files for applications/package managers
- /var/tmp/ – Temporary files (more persistent than /tmp)
- /var/lib/ – State info for services (e.g., dpkg, rpm, mysql)

Used by Services and Daemons
- Web servers (like Apache) store runtime files here.
- Package managers store metadata in /var.

Can Grow Large
- Monitor disk usage in /var – especially /var/log/.
- **Uncontrolled growth can fill up disk and crash services**.

Critical for System Health
- Deleting or mismanaging files here can break logging, updates, or daemons.

How logs work in Linux

The Big Picture

Linux has **two major systems** for logging:

1. **rsyslog** – The **traditional** logging daemon
2. **systemd-journald** – The **newer** logging system used with **journalctl**

Both are involved in **handling logs** — many of which end up in the **/var/log/** directory.

NOTE: "Think of systemd-journald as the **collector**, rsyslog as the **file writer**, and /var/log/ as the **library** where logs live."

1. rsyslog – The Traditional Log Writer

What it does:

Listens for log messages from the kernel, services, and apps.

Where it writes:

Writes logs as **text files under /var/log/**, such as:

- •/var/log/messages
- •/var/log/secure
- •/var/log/maillog

Configuration File (remember /etc folder!):

/etc/rsyslog.conf and /etc/rsyslog.d/

Example:

tail –f /var/log/messages

2. journalctl – The systemd Log Viewer

What it does:

Reads logs collected by **systemd-journald**, which captures logs **in binary format** (not plain text).

Where logs are stored:

- •/var/log/journal/ → Persistent storage if enabled
- •Or in memory (lost after reboot if not configured)

Command to view logs:

journalctl

Useful filters:

journalctl -u sshd # Logs for SSH service journalctl --since "1 hour ago"

How They Work Together

Component	Role	Storage Location
systemd-journald	Collects logs (new system)	/run/log/journal/ or /var/log/journal/
rsyslog	Processes + writes to text files	/var/log/
/var/log/	Directory where traditional logs live	Files created by rsyslog, cron, etc.
journalctl	Reads systemd-journald logs	Uses binary journal data

What is logrotate?

logrotate is a **log management tool**. It automatically:

- **Rotates** (renames) logs.
- **Compresses** old logs.
- **Deletes** logs older than X days.
- **Prevents /var/log from filling up.**

Practice: Basic logrotate Steps

Step 1: Check Existing Config

[root@myserver ~]# cat /etc/logrotate.conf

This is the **main config file**.

Step 2: See Service-Specific Configs

[root@myserver ~]# ls /etc/logrotate.d/

btmp chrony dnf firewalld kvm_stat rsyslog sssd wtmp

You will find **individual config files** for apps like rsyslog, httpd, yum, etc.

Step 3: Create a Cronjob (A scheduled Job) that grabs the logs and moves them to a Network File Share known as NFS

/proc

Kernel Level

- #Uptime
- # ls /proc/uptime

Sysadmin Commands most used in the Industry

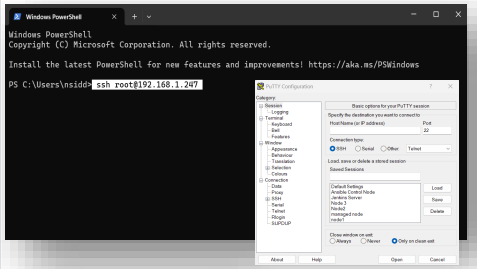
```
cat /proc/cpuinfo
cat /proc/meminfo
cat /proc/uptime
cat /proc/loadavg
cat /proc/mounts
ls /proc
cat /proc/1/cmdline
cat /proc/$$/status
```

SECURE SHELL (SSH)

Nexus Office

NOTE: All of these locations are REMOTE LOCATIONS

HOME

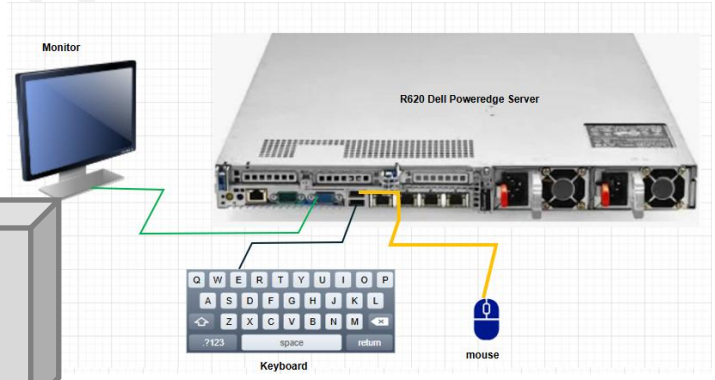


VPN, Tunnel, OpenVPN

- OpenVPN
- VPN
- Tunnel

SSH

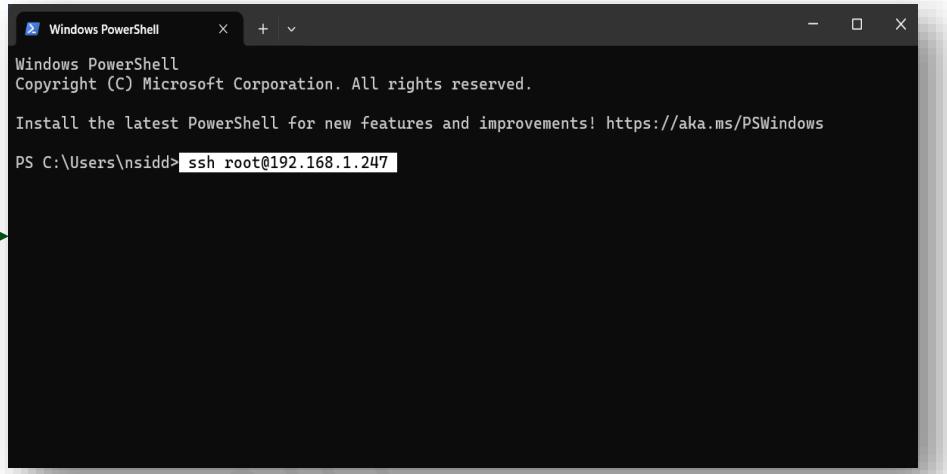
DATA CENTER
"on-prem"



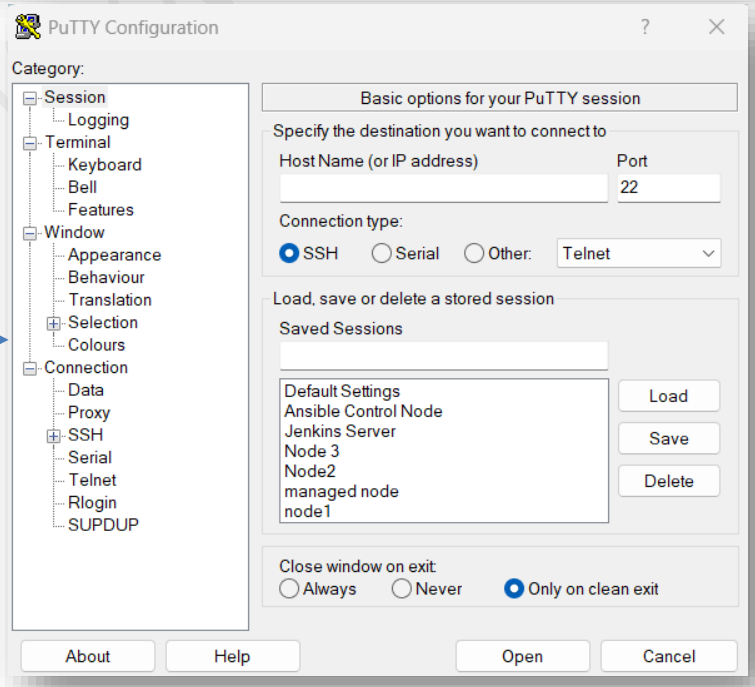
Client — Server

SSH – HOW MANY WAYS CAN YOU ACCESS A REMOTE SERVER

- Terminal
- CMD
- Windows Powershell



PuTTy



THANK YOU

Questions?